

CS 3563: Introduction to DBMS II

Assignment 1: Access Methods

Due Date and Time: 7 Feb. 2026, 11:59 PM IST

Instructions

Total Marks: 30

1. The assignment must be completed in the assigned groups.
2. All experiments must be performed on PostgreSQL version 14 or higher.
3. The submission must be a ZIP file containing all required files, organized according to the folder structure specified in the Submission Guidelines section, and submitted via Moodle.
4. Late submissions will be handled in accordance with the course policy.

Task 0: Database Setup and Configuration

0 Marks

Objective

Install PostgreSQL database management system and load the TPC-H `lineitem` table (TPC-H benchmark reference: https://www.tpc.org/tpc_documents_current_versions/pdf/tpc-h_v2.17.1.pdf).

PostgreSQL Installation

Install PostgreSQL version 14 or higher on your system. You may refer to the following resources:

- Official PostgreSQL Downloads: <https://www.postgresql.org/download/>
- Tutorial Reference: https://www.w3schools.com/postgresql/postgresql_install.php

After installation, verify that PostgreSQL is running correctly by:

1. Connecting to the default `postgres` database
2. Execute a simple query to verify functionality

TPC-H Database Setup

The TPC-H benchmark is a decision support benchmark that consists of a suite of business-oriented ad-hoc queries.

1. Follow the installation guide to create the `lineitem` table and load the CSV data:
https://drive.google.com/file/d/1gfDuE5X6b7XkNknduySz69JtdHLJ2fkS/view?usp=drive_link
2. Execute a simple query to verify functionality (e.g., `SELECT COUNT(*) FROM lineitem;`)

Task 1: Index Creation

(5 Marks)

Objective

Compare the total latency when (a) an index is created before data insertion versus (b) an index is created after data insertion. While creating the table, do *not* define any primary key constraint, as it may affect the experimental results.

Instructions:

1. You must use the `l_quantity` attribute as an indexed column for your experiments.
2. The `lineitem` table has already been set up during the installation guide. For this experiment, create two separate copies of that table corresponding to the two scenarios.
 - Scenario (a): Create the table and add the index before loading the data.
 - Scenario (b): Create the table, load the data, and then add the index afterward.
3. Measure and compare the total execution time for both scenarios (index before insertion and index after insertion).
4. State your observations by submitting:
 - A `query.sql` file containing all executed queries.
 - An analysis PDF file describing your observations and conclusions, along with screenshots of query execution and corresponding results.

Task 2: Index Usage

(5 + 10 Marks)

Part A: Query Parameter Computation for Target Selectivity

Objective: Implement a program that calculates the query parameter value for a given column and selectivity percentage.

Selectivity: Selectivity refers to the fraction of input rows that satisfy a given predicate condition. For example, a selectivity of 0.25 means that 25% of the total rows in the table satisfy the condition.

Problem Statement: Your program should accept two inputs: a column name and a target selectivity value (between 0 and 1). The program should calculate the query parameter value X such that the query `SELECT * FROM lineitem WHERE <column_name> <= X` returns the target percentage of rows from the table. In other words, given a desired selectivity, you need to find the value X (query parameter) that achieves this selectivity. A tolerance of 1% is accepted with respect to the resulting selectivity, i.e., the fraction of rows returned by the query may deviate by up to $\pm 1\%$ from the target selectivity.

If no parameter value achieves the target selectivity s within $\pm 1\%$ tolerance, report the parameter value whose corresponding selectivity is closest to s .

Instructions:

1. Implement the program in any programming language of your choice (Python, Java, C++, etc.)
2. The program should accept two inputs via command line arguments:
 - Column name (e.g., `l_quantity`)
 - Target selectivity (value between 0 and 1)
3. **Evaluation Command:** Your code will be executed as:

```
python3 task2A.py l_quantity 0.25
```

Similar execution commands will be used for other programming languages such as Java, C++, etc.

4. **Required Output Format:**

```
QUERY_PARAMETER: 13.0
QUERIES_EXECUTED: 5
ACTUAL_SELECTIVITY: 0.2498
```

These three lines must appear in your output.

5. Verify the actual selectivity achieved by your calculated query parameter.
6. Submit well-documented code with a README file.
7. **Code Template:** You may use the provided template as reference: https://drive.google.com/file/d/1kwy6YB3lrSJ1XNKdDDvhP6-fvqEPvnRe/view?usp=drive_link

Part B: Performance Analysis

Objective: Execute range queries with varying selectivity and analyze query performance across different indexing configurations. For the experiment, vary the *selectivity* in steps of **2% (0.02)** to obtain multiple selectivity levels. Specifically, for a given query template, 50 instances corresponding to selectivity values {0, 2%, 4%, ..., 100%} must be considered.

Important: You must use the *same program developed in Part A* to compute the query parameter values corresponding to the required selectivity levels. The computed query parameters must be carried forward and used consistently for all experiments in this part.

You must use the same query template described in Task 2 (Part A) for constructing the range queries. Further, assume **C** denotes the column on which the range filter predicate is applied.

Compare the following five scenarios:

- (a) Query execution without any index.
- (b) Query execution with an index on **C**.

- (c) Disable sequential scan operator and repeat (b).
- (d) Query execution with a clustered index on C.
- (e) Replace the `SELECT *` clause in the query template to be `SELECT C`, and execute with an index on C.

Instructions:

1. The attribute `l_quantity` must be used as the range predicate column for all queries, and the same attribute must be used for index creation wherever applicable.
2. Plot a single combined graph containing five line plots, one for each scenario. The X-axis should represent selectivity, and the Y-axis should represent query execution time (in milliseconds). **The X-axis must represent the actual selectivity achieved by the query and not the input target selectivity.**
3. Analyze and compare the performance behavior observed across all five scenarios, and explain the underlying reasons for the differences in execution time.

Deliverables:

- **Part A:**
 - Source code file (e.g., `task2A.py`).
 - Well-documented `README` file explaining the approach and usage.
- **Part B:**
 - Source code file for performance analysis (e.g., `task2B.py`).
 - Analysis report in PDF format containing graphs, observations, and explanations.
 - Query execution plans saved in JSON format.

Task 3: Index Selection

(10 Marks)

Objective

Analyze TPC-H Query 6 to identify opportunities for index selection. Create appropriate indexes and demonstrate performance improvements through comparative analysis.

Query Specification

```
SELECT SUM(l_extendedprice * l_discount) AS revenue
FROM lineitem
WHERE l_shipdate >= date '[DATE1]'
      AND l_shipdate < date '[DATE1]' + interval '1' year
      AND l_discount BETWEEN [DISCOUNT1] - 0.01 AND [DISCOUNT1] + 0.01
      AND l_quantity < [QUANTITY];
```

Resource: TPC-H Specification v2.17.1, Section 2.4 (Query Definitions).
Available at: https://www.tpc.org/tpc_documents_current_versions/pdf/tpc-h_v2.17.1.pdf

Instructions

1. Execute the query without any indexes and measure execution time.
2. Identify and enumerate all possible candidate search keys suitable for indexing that could improve query execution performance.
3. Explain which indexes you created as candidates, and which index would you recommend for this query instance.

Submission Guidelines

Folder Structure

Your submission must follow this exact structure:

```
<member1RollNo>-<member2RollNo/>

task1/
  analysis.pdf
  queries.sql

task2/
  partA/
    task2A.py (or .java, .cpp etc)
    README.md

  partB/
    analysis.pdf
    task2B.py (or .java, .cpp etc)
    query_plans/
      without_index.json
      with_index.json
      no_sequential_scan.json
      clustered_index.json
      select_column.json

task3/
  analysis.pdf
  queries.sql
  query_plans/
    without_indexes.json
    with_indexes01.json
    with_indexes02.json
    ...
```

Query Plan Format

All query plans must be exported in JSON format using:

```
EXPLAIN (ANALYZE, FORMAT JSON)
<your query here>;
```

Important Guidelines for Query Plans:

- Example of correct format:

```
[
  {
    "Plan": { ... },
    "Execution Time": 1035.372
  },
  {
    "Plan": { ... },
    "Execution Time": 2809.930
  }
]
```

- Validate your JSON files using an online validator

Analysis PDFs

Each analysis PDF must include:

- Clear graphs with proper labels and legends
- Detailed observations and explanations
- Performance metrics and timing comparisons
- Justification for design choices
- System configuration details (e.g., operating system, RAM, CPU, storage) along with the PostgreSQL configuration parameters modified (e.g., using `pgtune`).

Submission Process

1. Create a ZIP file of your entire submission folder
2. Name the ZIP file as: <member1RollNo>-<member2RollNo>.zip
3. Submit only the ZIP file via Moodle